Tablouri şi structuri

Tablouri - acces la elemente (1)

- adresa unui element al unui tablou este formată din
  - adresa de început a tabloului
  - deplasamentul elementului în cadrul tabloului
- cum pot fi exprimate sintactic?

Tablouri - acces la elemente (2)

- adresa de început a tabloului
  - prin numele variabilei tablou
- deplasamentul elementului în cadrul tabloului
  - constantă
  - registru
  - eventual combinații
  - nu o altă variablă din memorie

Diferență C - limbaj de asamblare

- în limbajul C, indicele exprimă deplasamentul în elemente ale tabloului
  - dimensiunea elementelor depinde de tipul lor
- în limbaj de asamblare se precizează deplasamentul în octeți
  - calculele trebuie făcute de către programator

Exemple (1)

- cod C

```
int t[100];
t[2]=5;
```

- traducere în limbaj de asamblare

```
mov t[8],5
```

- deoarece tipul `int` are 4 octeți, deplasamentul este $2 \cdot 4 = 8$

Exemple (2)

- cod C

```
int t[100],i;
// i are valoarea 3
t[i]=5;
```

- traducere în limbaj de asamblare

```
// eax are valoarea 3
mov t[eax*4],5
```

Variante sintactice (1)

```
mov t[eax*4],5
```

este echivalent sintactic cu

```
mov t+eax*4,5
```

- cât de complexă poate fi expresia deplasamentului?
- limitată de modurile de exprimare a adreselor în limbajul procesorului

Variante sintactice (2)

- cu ce înlocuiește compilatorul numele tabloului
  - tabloul este variabilă globală - adresa sa este o constantă
    - ex.: `[1000]`
  - tabloul este variabilă locală - adresa sa este epxrimată relativ la vârful stivei
    - ex.: `[ebp-4]`

Pointeri (1)

- în exemplul anterior, considerăm și o variabilă de tip pointer

```
int t[100];
int *p=t;
```

- în limbajul C, următoarele sunt echivalente

```
t[i]=5;
p[i]=5;
```

Pointeri (2)

- dar în limbaj de asamblare?
- NU

`mov p[eax*4],5` - greșit

- scriere corectă

```
mov ebx,p
mov dword ptr [ebx+eax*4],5
```

Tablouri ca parametri (1)

```
int t[100];
int *p=t;
void f(int x[]) {...}
void g(int *x) {...}
```

- ce tipuri de parametri acceptă fiecare dintre aceste funcții?

Tablouri ca parametri (2)

- toate combinațiile sunt permise

```
f(t);
g(t);
f(p);
g(p);
```

- de ce?

Tablouri ca parametri (3)

- indiferent de sintaxa folosită, pe stivă se depune un pointer ca parametru
- chiar dacă furnizăm un tablou ca parametru efectiv, pe stivă se depune adresa sa de început
  - conversie implicită
  - în implementarea funcției, accesul la parametru în limbaj de asamblare se face ca la un pointer

Tablouri bidimensionale (1)

- tablouri alocate static

```
int t[3][3];
```

- tablouri cu alocare dinamică

```
int **p;
```

- transmitere ca parametri

```
void f(int x[][3]) {...}
void g(int **x) {...}
```

Tablouri bidimensionale (2)

- apeluri valide

```
f(t);
g(p);
```

- apeluri invalide (erori de compilare)

```
f(p);
g(t);
```

- cauză - reprezentări diferite în memorie

Tablou alocat static

- reprezentare în memorie - liniară
- elementele sunt plasate în memorie la adrese consecutive, în ordinea

`t[0][0]`, `t[0][1]`, `t[0][2]`, `t[1][0]` etc.

- care este adresa elementului `t[i][j]`?
- deplasamentul în tablou: `i`·3+`j`

Tablou alocat dinamic (1)

- alocare dinamică

```
p=new int*[3];
for(i=0;i<3;i++)
  p[i]=new int[3];
```

- sunt create prin alocare
  - un tablou de 3 pointeri
  - 3 tablouri de câte 3 numere întregi

Tablou alocat dinamic (2)

- care este adresa elementului `p[i][j]`?
- folosind pointerul `p`, se accesează elementul `p[i]`
- folosind pointerul `p[i]`, se accesează elementul `p[i][j]`
- sunt necesari 2 paşi
  - complet diferit de modul de acces la tabloul alocat static

Structuri

- adresa unui câmp al unei structuri este formată din
  - adresa de început a structurii
  - deplasamentul câmpului în cadrul structurii
- similar tablourilor
- dar câmpurile unei structuri pot fi de tipuri diferite
  - deplasamentul nu mai este dat de un indice

Alinierea adreselor

- deplasamentul unui câmp - multiplu de dimensiunea sa
- consecință - pot exista goluri (octeți neutilizați) între câmpuri
- iar dimensiunea structurii poate fi mai mare decât suma dimensiunilor câmpurilor

Exemplu

```
struct S {
  char a,b;
  int c;
  } s;
```

- dimensiunea variabilei `s` de tip `S` - 8 octeți
- deplasamente: `a` - 0, `b` - 1, `c` - 4

Accesul la câmpuri (1)

- cod C

```
s.a=7;
s.c=22;
```

- traducere în limbaj de asamblare

```
mov byte ptr s,7
mov dword ptr s+4,22
// sau mov dword ptr s[4],22
```

Accesul la câmpuri (2)

- de ce trebuie precizată explicit dimensiunea operanzilor?
- folosim variabila `s`, a cărei dimensiune este de 8 octeți
- dar dorim să accesăm 1 octet, respectiv 4 octeți

Structuri ca parametri de funcții

- limbajul C nu permite transmiterea de structuri ca parametri ai unor funcții
  - doar pointeri la structuri
- în limbajul C++ este posibil
- cum traducem operatorul –> ?
  - determinarea deplasamentului
  - dereferențierea

Exemplu

- cod C

```
S *p=new S;
p->b=11;
```

- traducere în limbaj de asamblare

```
mov eax,p
mov byte ptr [eax+1],11
```